

BAB IV – PEMODELAN PROSES TINGKAT LANJUT

4.1. Capaian Dan Tujuan Pembelajaran

Setelah menempuh bab ini, mahasiswa mampu:

- Mahasiswa memahami konsep pengulangan dan koreksi lanjutan dalam pemodelan proses.
- Mahasiswa memahami cara menangani kejadian (*events*) dalam proses bisnis.
- Mahasiswa memahami pengembangan model proses yang lebih kompleks.

4.2. Pendahuluan

Sebagai kelanjutan dari pembahasan sebelumnya, pada bagian ini kita akan mendalami lebih jauh teknik pemodelan proses bisnis yang kompleks menggunakan BPMN. Konsep-konsep yang akan dijelaskan merupakan pengembangan dari pengetahuan yang telah diperoleh pada Bab 3. Fokus utama dalam pembahasan kali ini adalah perluasan konsep aktivitas, *events*, dan *gateways*.

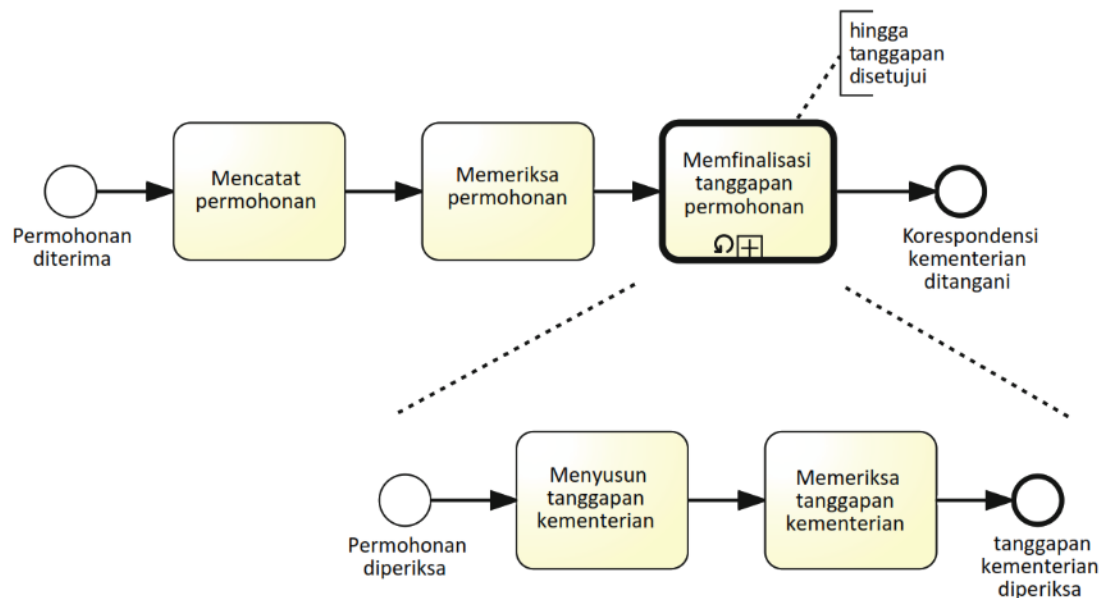
Aktivitas akan diperluas untuk memungkinkan pemodelan bentuk pengerjaan ulang (*rework*) dan pengulangan (*repetition*) yang lebih kompleks. Selain itu, akan dibahas jenis-jenis peristiwa yang lebih spesifik, seperti *message events*, *temporal events*, dan *cancellation events*. Peristiwa-peristiwa ini dapat digunakan untuk memodelkan *race conditions* dengan memanfaatkan jenis *gateways* baru.

Bagian ini juga akan membahas bagaimana pemanfaatan peristiwa dapat digunakan untuk menangani pengecualian (*exceptions*) dalam proses bisnis, sehingga model yang dihasilkan menjadi lebih realistis dan adaptif terhadap berbagai kemungkinan yang terjadi dalam praktik nyata.

4.3. Pengerjaan ulang (*rework*) dan Pengulangan (*repetition*)

Pada bab sebelumnya, telah dijelaskan bagaimana memodelkan *rework* dan *repetition* melalui *XOR-gateways*. Sub-proses yang diperluas (*expanded sub-processes*) menawarkan cara alternatif untuk memodelkan bagian dari proses yang dapat diulang. Mari kita tinjau kembali proses penanganan korespondensi kementerian pada Contoh 3.7. Untuk menyederhanakan model tersebut, kita dapat mengambil fragmen yang dibatasi oleh XOR-join dan XOR-split (yang mencakup blok pengulangan dan cabang *loopback*) dan menggantinya dengan sebuah sub-proses yang berisi aktivitas-aktivitas

dalam blok pengulangan. Untuk menunjukkan bahwa sub-proses ini dapat diulang (jika tanggapan belum disetujui), kita menandai aktivitas sub-proses tersebut dengan simbol loop, seperti yang ditunjukkan pada Gambar 4.1.



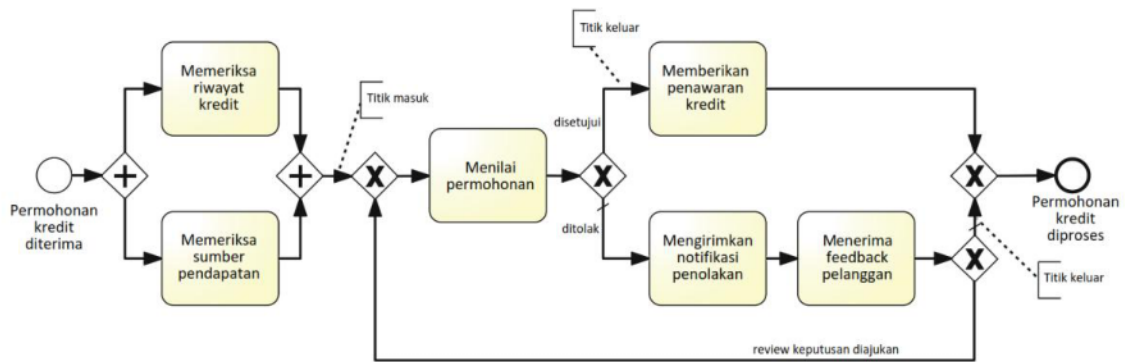
Gambar 4. 1 Model proses untuk menangani korespondensi kementerian pada Gambar 3.12 yang disederhanakan menggunakan aktivitas *loop*.

Kita juga dapat menambahkan anotasi untuk menyatakan kondisi pengulangan, misalnya: “hingga tanggapan disetujui”.

Seperti halnya sub-proses lainnya, Anda dapat memilih untuk tidak merinci isi dari sub-proses berulang tersebut. Namun, jika Anda memutuskan untuk merincinya, jangan lupa untuk menyertakan aktivitas pengambilan keputusan sebagai aktivitas terakhir dalam sub-proses. Tanpa aktivitas ini, tidak ada mekanisme yang dapat menentukan kapan sub-proses tersebut harus diulang.

Pertanyaan: Aktivitas *Loop* atau Siklus?

Aktivitas *loop* merupakan notasi ringkas untuk menggambarkan sebuah siklus terstruktur, yaitu blok pengulangan yang dibatasi oleh satu titik masuk dan satu titik keluar, seperti pada contoh dalam Gambar 4.1. Namun, dalam beberapa kasus, mungkin terdapat lebih dari satu titik masuk dan keluar, atau titik masuk maupun keluar berada di dalam blok pengulangan.



Gambar 4. 2 Contoh siklus tidak terstruktur

Sebagai contoh, perhatikan model pada Gambar 4.2. Dalam model tersebut, terdapat sebuah siklus yang terdiri dari aktivitas “Menilai permohonan”, “Memberi pemberitahuan penolakan”, dan “Menerima umpan balik dari pelanggan”. Siklus ini memiliki satu titik masuk dan dua titik keluar, sehingga dikategorikan sebagai siklus tak terstruktur (*unstructured cycle*).

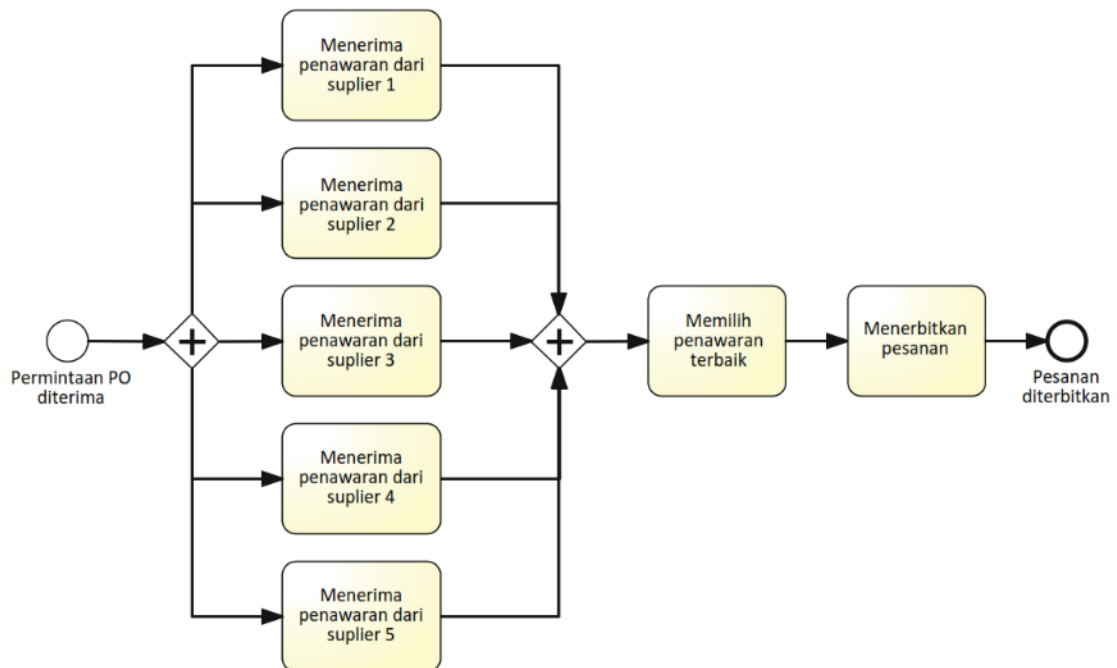
Siklus dengan lebih dari satu titik keluar, seperti dalam contoh ini, tidak dapat diubah menjadi siklus dengan satu titik keluar saja, kecuali jika ditambahkan kondisi-kondisi tertentu yang menjelaskan situasi-situasi di mana siklus dapat dihentikan.

4.3.1. Pengulangan Paralel

Aktivitas *loop* memungkinkan kita untuk menangkap pengulangan secara berurutan, artinya setiap instansi dari aktivitas tersebut dijalankan satu per satu. Namun, dalam beberapa situasi, kita mungkin perlu menjalankan beberapa instance dari aktivitas yang sama secara bersamaan, seperti pada contoh berikut.

Contoh 4.1

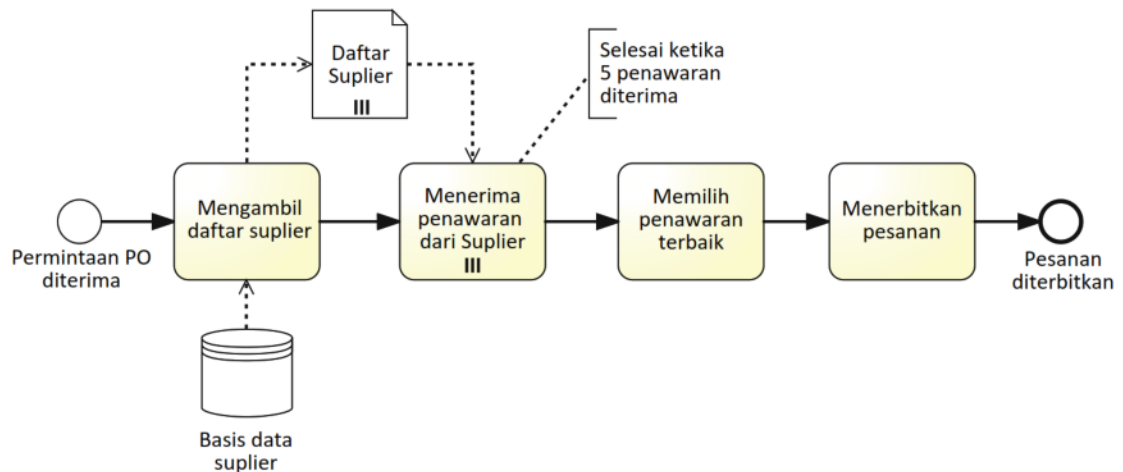
Dalam sebuah proses pengadaan, penawaran harga harus diperoleh dari semua pemasok yang telah dipilih. Setelah seluruh penawaran diterima, penawaran-penawaran tersebut dievaluasi dan penawaran terbaik dipilih. Setelah itu, pesanan pembelian yang sesuai akan dilakukan.



Gambar 4. 3 Menerima penawaran dari lima suplier

Misalkan terdapat lima pemasok utama. Kita dapat menggunakan gerbang AND-split untuk memodelkan lima tugas yang berjalan secara paralel, masing-masing untuk memperoleh penawaran dari satu pemasok, sebagaimana ditunjukkan pada Gambar 4.3. Namun, terdapat dua permasalahan pada solusi ini. Pertama, semakin banyak jumlah pemasok, maka semakin besar pula model yang dihasilkan, karena diperlukan satu tugas untuk setiap pemasok. Kedua, model perlu diperbarui setiap kali jumlah pemasok berubah. Dalam praktiknya, daftar pemasok sering kali diperbarui secara dinamis dan disimpan dalam basis data organisasi yang akan diakses terlebih dahulu sebelum menghubungi para pemasok.

Untuk mengatasi permasalahan tersebut, BPMN menyediakan sebuah konstruksi yang disebut Aktivitas *multi-instance*. Aktivitas *multi-instance* menunjukkan sebuah aktivitas (baik berupa tugas maupun sub-proses) yang dijalankan beberapa kali secara bersamaan, yakni secara paralel. Konstruksi ini sangat berguna ketika aktivitas yang sama harus dijalankan untuk beberapa entitas atau item data, seperti pada contoh permintaan penawaran dari banyak pemasok, pengecekan ketersediaan setiap item dalam suatu pesanan secara terpisah, pengiriman dan pengumpulan kuesioner dari sejumlah saksi dalam proses klaim asuransi, dan sebagainya.



Gambar 4. 4 Memperoleh penawaran dari sejumlah pemasok yang ditentukan secara dinamis

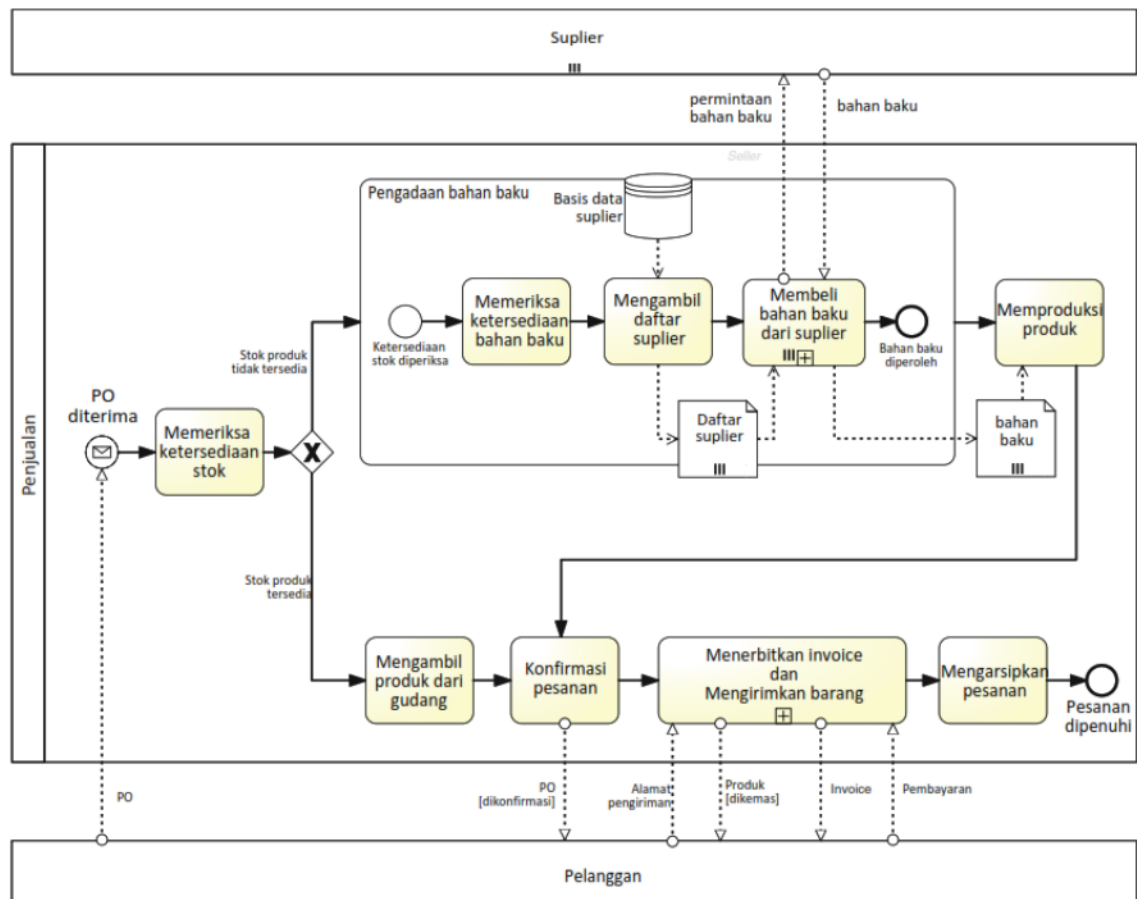
Sebuah aktivitas *multi-instance* digambarkan sebagai aktivitas yang diberi tanda tiga garis vertikal kecil di bagian bawahnya. Gambar 4.4 menunjukkan versi yang telah diperbarui dari model proses pengadaan pada Gambar 4.3. Model ini tidak hanya lebih ringkas, tetapi juga dapat bekerja dengan daftar suplier yang bersifat dinamis, yang dapat berubah pada setiap *instance*-nya. Untuk itu, ditambahkan sebuah tugas untuk mengambil daftar pemasok, kemudian daftar tersebut diteruskan ke aktivitas *multi-instance* yang bertugas menghubungi berbagai suplier tersebut.

Anda mungkin memperhatikan bahwa dalam contoh ini, objek data Daftar Suplier juga diberi simbol *multi-instance*. Hal ini digunakan untuk menunjukkan bahwa objek data tersebut merupakan koleksi dari data yang serupa, seperti daftar item pesanan atau daftar pelanggan. Ketika sebuah koleksi digunakan sebagai masukan untuk aktivitas *multi-instance*, jumlah item dalam koleksi tersebut akan menentukan berapa banyak *instance* aktivitas yang akan dibuat. Sebagai alternatif, jumlah *instance* yang akan dibuat juga dapat ditentukan melalui anotasi pada aktivitas *multi-instance* (misalnya, “15 suplier” atau “sesuai basis data suplier”).

Kita kembali pada contoh sebelumnya. Misalkan daftar suplier telah bertambah besar seiring waktu, katakanlah terdapat 20 suplier dalam basis data. Namun, sesuai kebijakan organisasi, lima penawaran dari lima suplier yang berbeda sudah cukup untuk mengambil keputusan. Oleh karena itu, kita tidak ingin menunggu semua 20 suplier merespons permintaan penawaran. Untuk mengatasinya, kita dapat menambahkan anotasi pada aktivitas *multi-instance* yang menyatakan jumlah minimum *instance* yang

harus diselesaikan sebelum kontrol dialihkan ke busur keluar (misalnya, “selesai jika 5 penawaran telah diterima”, seperti yang ditunjukkan pada Gambar 4.4).

Ketika aktivitas *multi-instance* dijalankan, 20 token akan dihasilkan, masing-masing menandai kemajuan dari satu instansi aktivitas. Kemudian, segera setelah lima *instance* pertama selesai, seluruh *instance* lainnya akan dibatalkan (token masing-masing akan dihancurkan), dan satu token dikirimkan ke busur keluar sebagai penanda penyelesaian aktivitas.



Gambar 4. 5 Menggunakan kolom multi-instance untuk merepresentasikan beberapa suplier

Mari kita ambil contoh *order-to-cash* pada Gambar 3.17, dan perluas isi dari sub-proses pengadaan bahan baku. Untuk membuat model ini lebih realistis, kita dapat menggunakan sub-proses *multi-instance* sebagai pengganti struktur yang dibatasi oleh dua gerbang OR, dengan asumsi bahwa daftar pemasok yang akan dihubungi akan ditentukan secara dinamis dari basis data suplier (model yang telah diperbarui ditampilkan pada Gambar 4.5). Dengan prinsip yang sama, kita mengganti dua kolom “Suplier 1” dan “Suplier 2” menjadi satu kolom tunggal bernama “Suplier”, yang juga

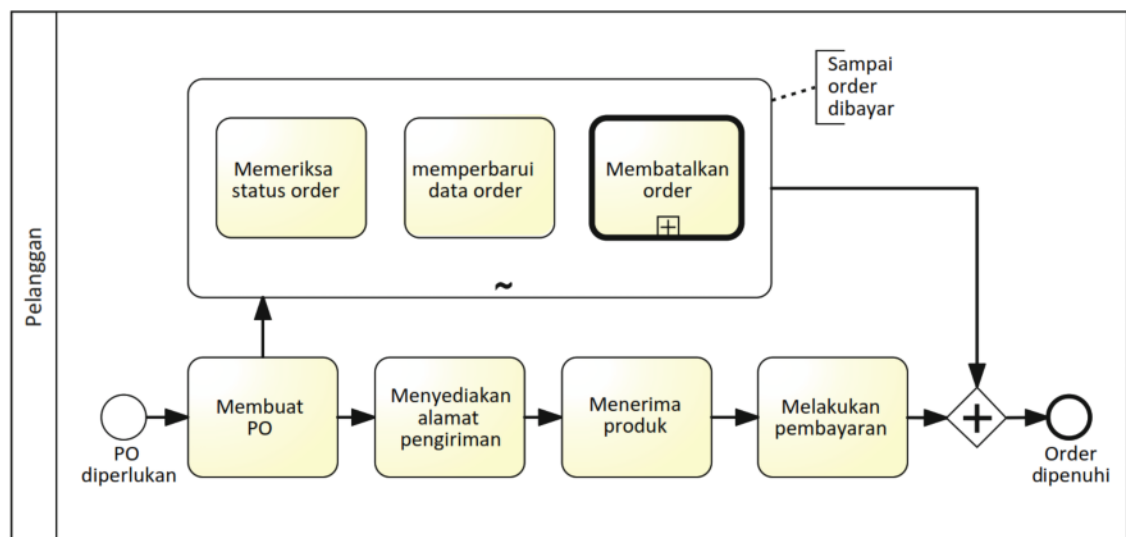
diberi simbol *multi-instance*. Kolam *multi-instance* ini merepresentasikan sekumpulan kelas sumber daya atau sumber daya yang memiliki karakteristik serupa.

Dari gambar tersebut, dapat diamati bahwa terdapat empat alur pesan yang terhubung ke sub-proses “Menerbitkan invoice dan Mengirimkan barang” sebagai hasil dari bentuk *collapsed* pada aktivitas tersebut. Urutan pertukaran pesan ditentukan oleh aktivitas-aktivitas di dalam sub-proses yang menerima dan mengirimkan pesan tersebut.

4.3.2. Pengulangan Tak Terkendali

Terkadang kita perlu memodelkan satu atau lebih aktivitas yang dapat diulang beberapa kali tanpa urutan tertentu hingga suatu kondisi terpenuhi. Misalnya, dalam proses *order-to-cash*, pelanggan mungkin ingin menanyakan perkembangan pesanan mereka. Pelanggan dapat melakukan hal ini dengan mengirimkan email kepada penjual. Hal ini dapat dilakukan kapan saja setelah pelanggan mengirimkan pesanan pembelian, dan sebanyak yang diinginkan.

Demikian pula, pelanggan dapat mencoba membatalkan pesanan atau memperbarui data pribadi sebelum pesanan tersebut dipenuhi. Aktivitas-aktivitas ini bersifat tidak terkendali karena dapat terjadi berulang kali, tanpa urutan tertentu, atau bahkan tidak dilakukan sama sekali, sampai suatu kondisi terpenuhi—dalam hal ini, pesanan telah dipenuhi.



Gambar 4. 6 Menggunakan sub-proses *ad hoc* untuk memodelkan pengulangan yang tidak terkendali

Untuk memodelkan sekumpulan aktivitas tak terkendali, kita dapat menggunakan sub-proses *ad hoc*. Gambar 4.6 menunjukkan contoh proses pelanggan, di mana kondisi

penyelesaian ("hingga pesanan terpenuhi") ditentukan melalui anotasi. Sub-proses *ad hoc* ditandai dengan simbol tilde (~) di bagian bawah kotak sub-proses.

Urutan parsial antaraktivitas dalam sub-proses *ad hoc* dapat ditentukan melalui *sequence flow*. Namun, kita tidak dapat merepresentasikan *start event* dan *end event* dalam sub-proses jenis ini.

4.4. Penanganan *Event*

Kita telah mempelajari pada Bab 3 bahwa *event* digunakan untuk memodelkan sesuatu yang terjadi secara seketika dalam suatu proses. Kita telah melihat *start event* (peristiwa awal), yang menandai bagaimana sebuah *instance* proses dimulai (*token* dibuat), dan *end event* (peristiwa akhir), yang menandai kapan sebuah *instance* proses berakhir (*token* dihancurkan).

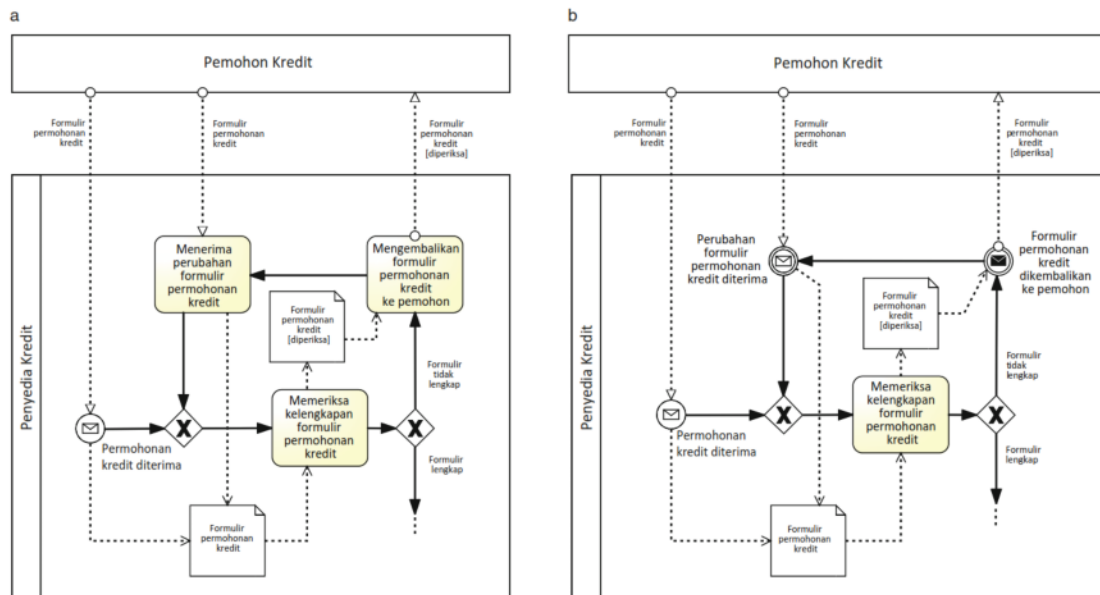
Ketika suatu *event* terjadi di tengah jalannya proses—misalnya, konfirmasi pesanan diterima setelah pesanan dikirimkan ke pelanggan dan sebelum proses pengiriman dilakukan—peristiwa tersebut disebut sebagai *intermediate event* (peristiwa antara). Sebuah *token* akan tertahan di *sequence flow* masuk dari *intermediate event* hingga peristiwa tersebut terjadi. Setelah peristiwa terjadi, *token* akan langsung melewati *event* tersebut secara seketika, karena *event* tidak dapat menyimpan *token*.

Intermediate event direpresentasikan dalam BPMN sebagai lingkaran dengan garis ganda di sekelilingnya.

4.5.1. *Message Events*

Pada bab sebelumnya, telah dijelaskan bahwa kita dapat menandai sebuah *start event* dengan simbol amplop kosong untuk menunjukkan bahwa *instance* proses baru dipicu oleh diterimanya sebuah pesan (lihat Gambar 3.15). Selain *start message event*, kita juga dapat menandai *end event* dan *intermediate event* dengan simbol amplop untuk merepresentasikan interaksi antara proses kita dan pihak lain. Jenis-jenis peristiwa ini secara kolektif disebut sebagai *message events*.

Sebuah *end message event* menandakan bahwa proses berakhir dengan pengiriman pesan. Sedangkan *intermediate message event* menandakan bahwa pesan diterima atau dikirim selama pelaksanaan proses. *Intermediate message event* dan *end message event* merupakan notasi alternatif terhadap aktivitas-aktivitas yang semata-mata digunakan untuk mengirim atau menerima pesan.



Gambar 4. 7 Mengganti aktivitas yang hanya mengirim atau menerima pesan (a) dengan *message event* (b)

Sebagai contoh, perhatikan aktivitas “Mengembalikan formulir permohonan kredit ke pemohon pemohon” dan “Menerima perubahan formulir permohonan kredit” pada Gambar 4.7a, yang merupakan cuplikan dari model penilaian pinjaman. Akan lebih bermakna jika aktivitas pertama digantikan dengan *intermediate send message event*, dan aktivitas kedua digantikan dengan *intermediate receive message event*, sebagaimana ditunjukkan pada Gambar 4.7b, karena aktivitas-aktivitas tersebut tidak benar-benar merepresentasikan satuan kerja, melainkan hanya pengiriman atau penerimaan pesan secara mekanis.

Sebuah *intermediate message event* yang menerima pesan digambarkan seperti *start message event*, tetapi dengan garis ganda. Jika *intermediate event* menandakan pengiriman pesan, maka simbol amplop digambarkan berwarna gelap.

Selain itu, jika sebuah aktivitas pengiriman pesan langsung diikuti oleh *end event* tanpa jenis (*untyped*), maka keduanya dapat digantikan dengan *end message event*, karena aktivitas tersebut semata-mata digunakan untuk mengirim pesan sebelum proses berakhir. *End message event* digambarkan sebagai end event dengan amplop gelap.

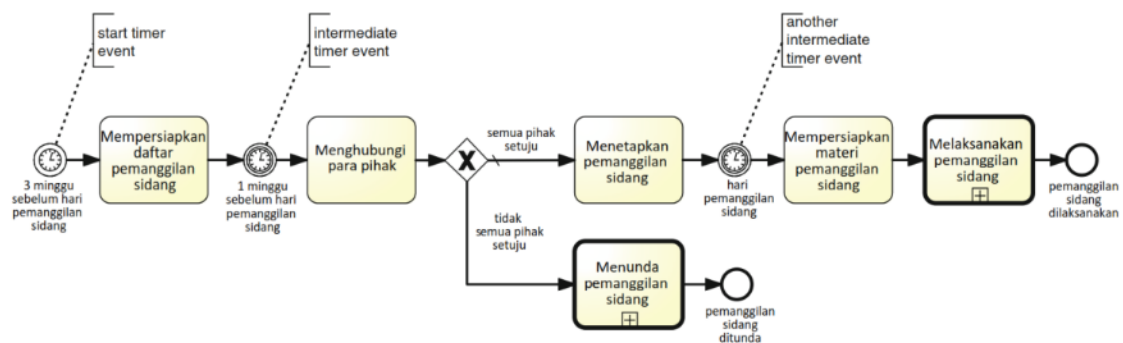
Perlu diperhatikan bahwa *start message event* bukan merupakan notasi alternatif dari *untyped start event* yang diikuti oleh aktivitas penerimaan pesan. Kedua konstruksi ini tidak dapat dipertukarkan. Pada kasus pertama, *instance* proses dimulai hanya setelah pesan tertentu diterima; sedangkan pada kasus kedua, *instance* proses dapat dimulai kapan saja, dan pesan akan diterima setelah aktivitas pertama dijalankan.

Pertanyaan: *Event* bertipe atau tidak bertipe?

Kami menyarankan untuk menentukan tipe suatu peristiwa apabila tipe tersebut diketahui, karena hal ini membantu pembaca dalam memahami model proses secara lebih baik.

4.5.2. Temporal Events

Selain *message event*, terdapat pemicu lain yang dapat ditentukan untuk *start event*. Salah satunya adalah *timer event* (kejadian waktu). Jenis kejadian ini menunjukkan bahwa instansi proses dimulai ketika suatu kejadian temporal tertentu terjadi, misalnya setiap Jumat pagi, setiap hari kerja dalam sebulan, atau setiap pagi pukul 7. *Timer event* juga dapat digunakan sebagai *intermediate event* (kejadian antara) untuk menunjukkan bahwa suatu interval waktu harus berlalu sebelum instansi proses dapat berlanjut. Untuk menunjukkan *timer event*, simbol kejadian ditandai dengan gambar jam tangan di dalam lingkaran. *Timer event* hanya bersifat *catching* (menangkap kejadian) karena pemicu waktu merupakan sesuatu yang berada di luar kendali proses. Dengan kata lain, proses tidak menghasilkan pemicu waktu, melainkan meresponsnya. Proses ini dikendalikan oleh tiga kejadian temporal: dimulai tiga minggu sebelum tanggal *callover*, berlanjut satu minggu sebelum tanggal *callover*, dan berakhir pada hari *callover*. Untuk memodelkan kejadian-kejadian temporal ini, kita memerlukan satu *start event* dan dua *intermediate timer event*, sebagaimana ditunjukkan dalam Gambar 4.8. Mari kita lihat bagaimana proses ini berjalan dari sudut pandang *token semantics* (semantik token).



Gambar 4. 8 Menggunakan *timer event* untuk mengendalikan berbagai aktivitas proses bisnis

Contoh 4.2

Perhatikan proses berikut yang berlangsung di sebuah persidangan perdata di pengadilan.

Di persidangan perdata, pemanggilan sidang dilaksanakan sekali dalam sebulan untuk menetapkan jadwal sidang perkara-perkara yang akan disidangkan. Proses untuk mempersiapkan pemanggilan sidang dimulai tiga minggu sebelum hari pemanggilan

sidang, yaitu dengan menyusun daftar pemanggilan sidang yang memuat informasi seperti detail kontak para pihak yang terlibat dan perkiraan tanggal sidang.

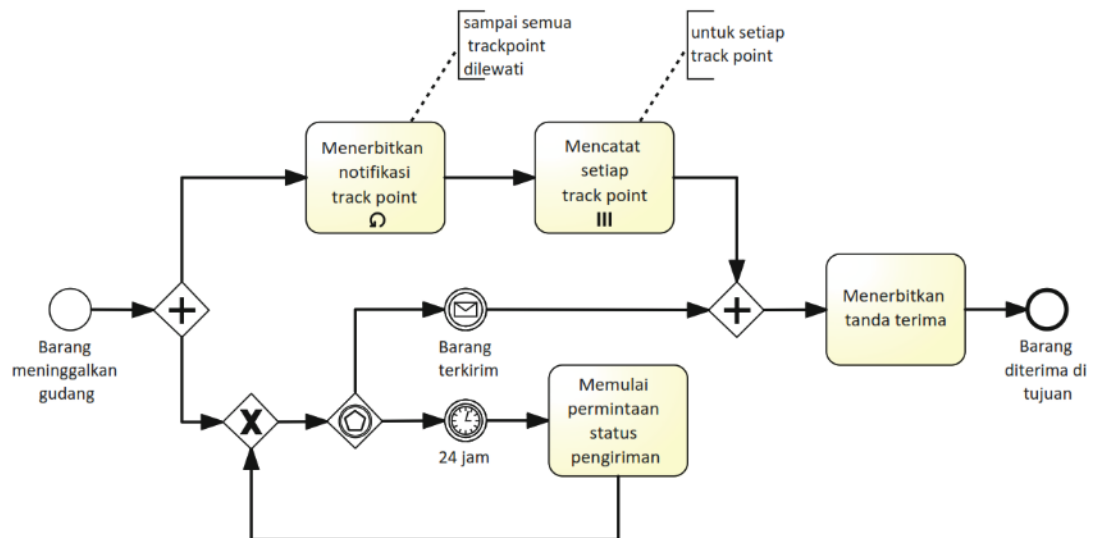
Satu minggu sebelum hari pemanggilan sidang, para pihak yang terlibat dihubungi untuk memastikan apakah mereka semua telah siap untuk menjalani persidangan. Jika seluruh pihak telah siap, maka pemanggilan sidang akan dijadwalkan. Namun, jika belum, maka proses pemanggilan sidang akan ditunda hingga slot waktu berikutnya yang tersedia.

Akhirnya, pada hari pelaksanaan pemanggilan sidang, bahan-bahan pendukung pemanggilan sidang disiapkan dan pemanggilan sidang pun dilaksanakan.

Sebuah token yang menangkap *instance* baru akan dihasilkan tiga minggu sebelum hari pemanggilan sidang (dengan asumsi bahwa hari telah dijadwalkan oleh proses lain). Setelah aktivitas pertama “Menmpersiapkan daftar pemanggilan sidang” selesai, token dikirim melalui anak panah masuk ke *intermediate timer event* berikutnya, yaitu “1 minggu sebelum hari pemanggilan sidang”. Kejadian ini kemudian menjadi aktif. Token tetap terperangkap di anak panah ini hingga kejadian temporal tersebut terjadi, yaitu hanya ketika tinggal satu minggu sebelum hari pemanggilan sidang. Setelah itu terjadi, token langsung melewati simbol *event* dan bergerak ke anak panah keluar. Inilah sebabnya mengapa *event* dianggap bersifat seketika: kejadian tidak dapat menahan token, berbeda dengan aktivitas yang menahan token selama durasi pelaksanaannya (ingat bahwa aktivitas memerlukan waktu untuk diselesaikan).

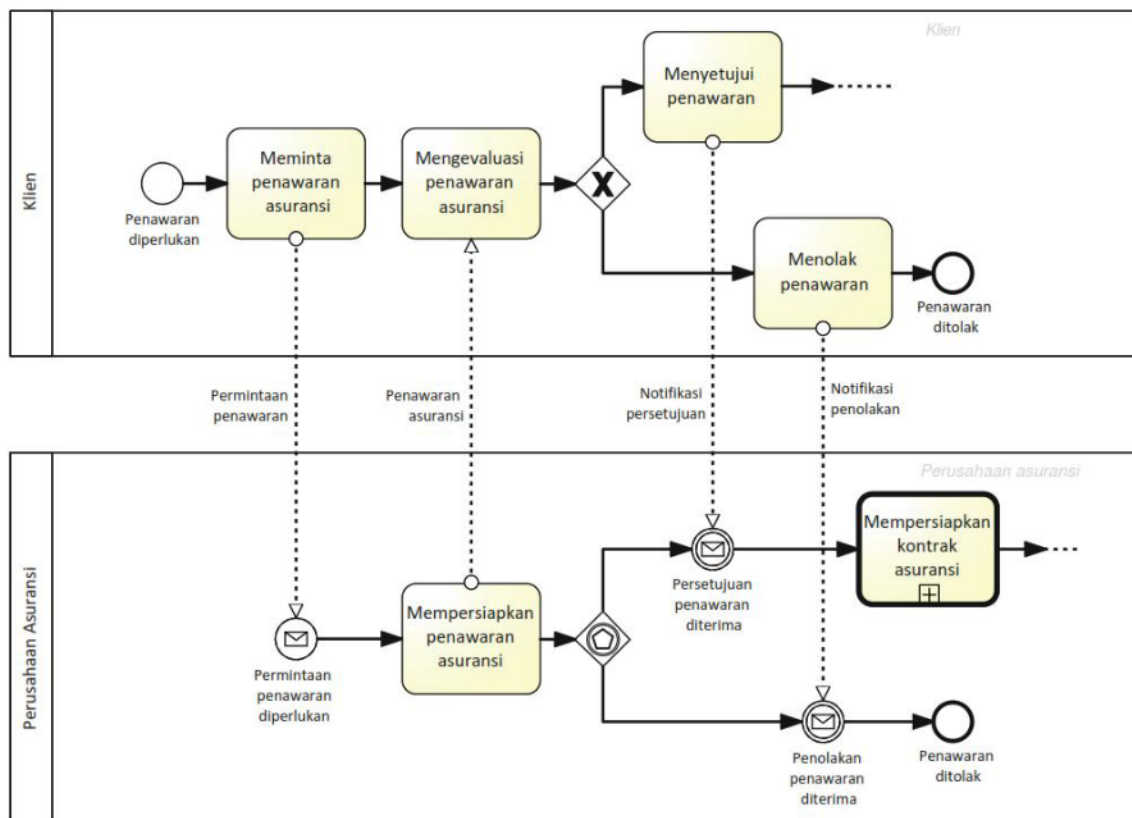
4.5.3. Racing Events

Salah satu skenario umum yang ditemui ketika memodelkan proses dengan kejadian (*event*) adalah ketika dua kejadian eksternal "berlomba" satu sama lain. Kejadian yang terjadi terlebih dahulu akan menentukan kelanjutan proses. Sebagai contoh, setelah sebuah penawaran asuransi dikirimkan kepada klien, klien dapat membalas dengan pesan penerimaan (dalam hal ini kontrak asuransi akan dibuat), atau dengan penolakan (sehingga penawaran tersebut akan dibuang).



Gambar 4. 9 Kondisi *race events* pada *message event* dan *timer*

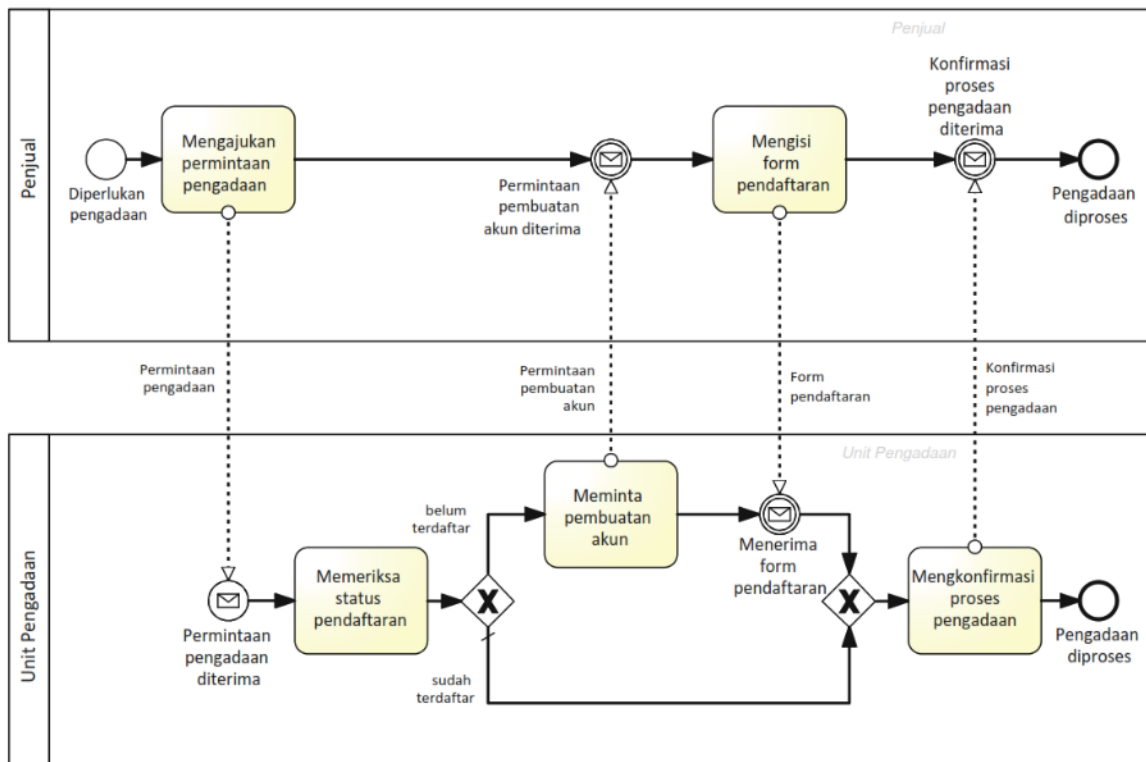
Perlombaan antara kejadian-kejadian eksternal ini dimodelkan melalui pemisahan eksklusif berbasis kejadian (*event-based exclusive split*). Pemisahan eksklusif berbasis kejadian ini direpresentasikan oleh gateway yang ditandai dengan lambang **segi lima kosong** yang dikelilingi lingkaran garis ganda. Gambar 4.9 memperlihatkan contoh pemisahan eksklusif berbasis kejadian. Ketika pelaksanaan proses mencapai titik ini (dengan kata lain, ketika sebuah *token* mencapai *gateway* ini), pelaksanaan akan berhenti hingga salah satu dari *message event* atau *timer event* terjadi. Kejadian yang lebih dahulu terjadi akan menentukan jalur mana yang akan dilanjutkan. Jika *timer event* yang terjadi terlebih dahulu, maka proses permintaan status pengiriman akan dimulai dan aliran eksekusi akan kembali ke *event-based gateway*. Jika pesan yang menandakan bahwa barang terkirim terlebih dahulu, maka aliran eksekusi akan berlanjut mengikuti alur menuju AND-join.



Gambar 4. 10 Mencocokkan pilihan internal di satu pihak dengan pilihan berbasis peristiwa di pihak lain

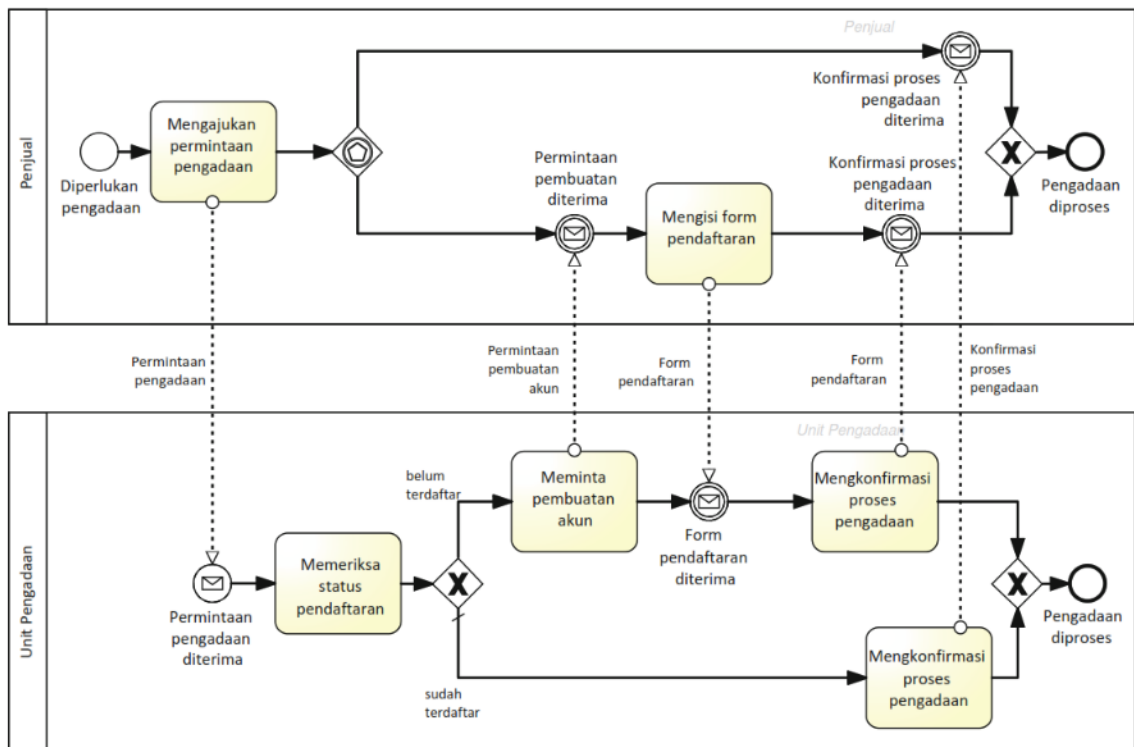
Perbedaan antara XOR-split yang telah kita bahas dalam Bab 3 dengan XOR-split berbasis kejadian adalah bahwa yang pertama memodelkan pilihan internal yang ditentukan oleh hasil dari aktivitas pengambilan keputusan, sedangkan yang kedua memodelkan pilihan yang ditentukan oleh lingkungan eksternal dari proses. Oleh karena itu, XOR-split dalam Bab 3 disebut data-based XOR-split, karena cabang yang akan diambil ditentukan berdasarkan evaluasi dari satu atau lebih kondisi pada data yang dihasilkan oleh aktivitas pengambilan keputusan. Dengan demikian, XOR-split berbasis kejadian hanya dapat diikuti oleh kejadian *catching* antara seperti *timer event*, *message event*, atau aktivitas penerimaan. Karena pilihan ditunda hingga suatu kejadian benar-benar terjadi, *event-based split* ini juga dikenal dengan istilah *deferred choice* (pilihan yang ditunda). Tidak ada *event-based XOR-join*, sehingga cabang-cabang yang keluar dari *event-based split* digabungkan kembali menggunakan XOR-join biasa. *Event-based gateway* juga dapat digunakan sebagai pasangan dari keputusan internal pada pihak yang berkolaborasi. Sebagai contoh, perhatikan Gambar 4.10. Pilihan yang dibuat dari dalam *pool* klien untuk mengirim notifikasi persetujuan atau penolakan kepada perusahaan asuransi harus dicocokkan dengan keputusan berbasis kejadian

pada *pool* perusahaan asuransi untuk merespons pilihan yang dibuat oleh klien tersebut.



Gambar 4. 11 Contoh kolaborasi yang dapat menemui jalan buntu jika keputusan dibuat untuk “sudah terdaftar”

Event-based gateway juga dapat digunakan untuk menghindari anomali perilaku dalam komunikasi antar pool. Ambil contoh diagram kolaborasi antara unit pengadaan dan penjual pada Gambar 4.11. Kolaborasi ini bisa mengalami *deadlock* jika penjual telah terdaftar, karena pihak ini akan menunggu pesan permintaan pembuatan akun yang tidak akan pernah dikirim. Untuk memperbaiki masalah ini, kita perlu mengizinkan penjual menerima pesan konfirmasi pembuatan akun secara langsung jika memang penjual sudah terdaftar, sebagaimana ditunjukkan dalam Gambar 4.12.



Gambar 4. 12 Menggunakan gateway berbasis peristiwa untuk memperbaiki masalah kebuntuan potensial dalam kolaborasi Gambar 4.11

Saat menghubungkan *pool* satu sama lain melalui *message flow*, pastikan Anda memeriksa urutan koneksi tersebut agar tidak terjadi *deadlock*. Ingat khususnya bahwa keputusan internal dari satu pihak perlu dicocokkan dengan keputusan berbasis kejadian dari pihak lainnya, dan bahwa aktivitas dengan *message flow* keluar akan mengirimkan pesan setelah aktivitas selesai, sementara aktivitas dengan *message flow* masuk akan menunggu pesan tersebut untuk dapat dimulai.

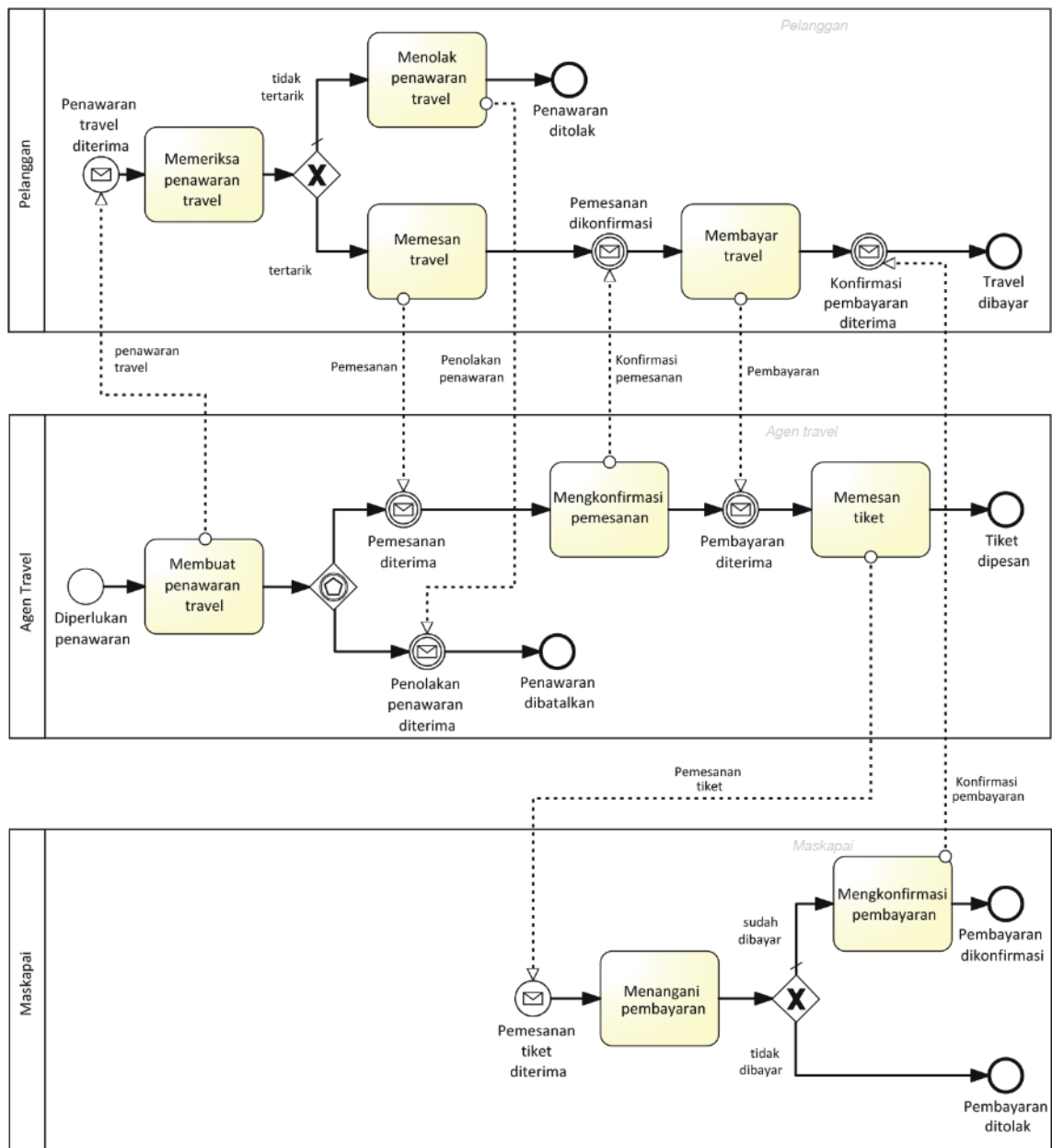
4.5. Menangani Pengecualian (*Handling Exceptions*)

Pengecualian (*exceptions*) adalah kejadian-kejadian yang menyebabkan suatu proses menyimpang dari jalur normalnya, yaitu dari apa yang biasa dikenal sebagai skenario "cuaca cerah" (*sunny-day scenario*). Skenario "cuaca buruk" (*rainy-day scenario*) sering kali terjadi dalam kenyataan, dan oleh karena itu perlu dimodelkan jika tujuannya adalah untuk mengidentifikasi semua kemungkinan penyebab masalah dalam suatu proses tertentu.

Pengecualian mencakup kesalahan bisnis seperti produk yang kehabisan stok atau telah dihentikan, serta kesalahan teknologi seperti kerusakan basis data, gangguan jaringan, atau pelanggaran logika program. Pengecualian-pengecualian ini dapat menyebabkan gangguan sementara atau penghentian total pada proses yang sedang berjalan. Sebagai contoh, dalam kasus produk yang kehabisan stok, proses order-to-cash mungkin perlu dihentikan sementara untuk memesan produk dari pemasok, atau bahkan dibatalkan sepenuhnya jika produk tersebut tidak dapat disediakan dalam jangka waktu yang telah ditentukan.

4.5.1. *Penghentian Proses*

Cara paling sederhana untuk menangani pengecualian adalah dengan menghentikan proses yang sedang berjalan dan memberikan sinyal bahwa proses berakhir secara tidak normal. Hal ini dapat dilakukan dengan menggunakan *end terminate event* (peristiwa akhir terminasi), seperti ditunjukkan pada Gambar 4.14. *End terminate event* (digambarkan sebagai peristiwa akhir dengan lingkaran penuh di dalamnya) menyebabkan penghentian langsung atas instansi proses saat ini dan seluruh sub-prosesnya.



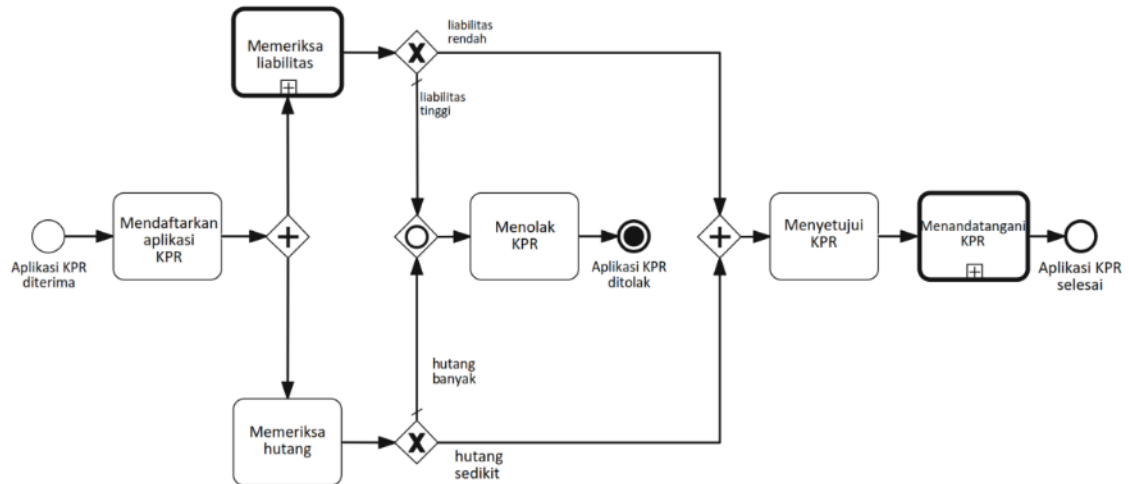
Gambar 4. 13 Diagram kolaborasi antara pelanggan, agen perjalanan, dan maskapai penerbangan

Dalam contoh pada Gambar 4.14 aplikasi KPR ditolak dan proses dihentikan jika pemohon memiliki utang atau kewajiban yang tinggi. *Event terminate* menghancurkan semua *token* dalam model proses dan sub-prosesnya. Dalam contoh ini, hal ini diperlukan untuk menghindari *deadlock* pada AND-join, karena token dapat terjebak sebelum AND-join jika ada salah satu kondisi utang tinggi atau kewajiban tinggi.

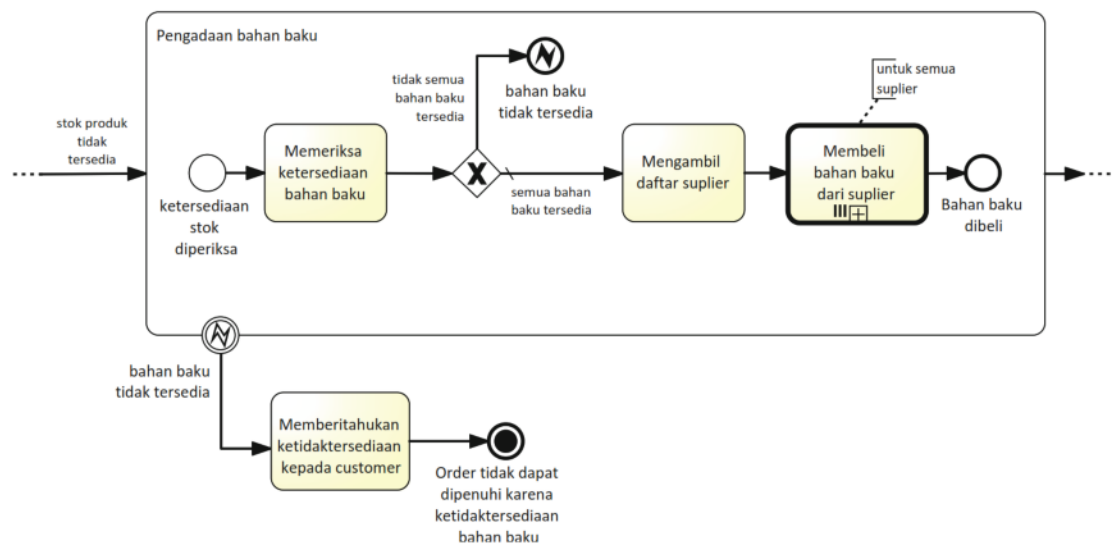
Perlu diperhatikan bahwa jika terminate event dipicu dari dalam suatu sub-proses, maka hanya sub-proses tersebut yang dihentikan, bukan proses induknya. Terminate event hanya menyebar ke bawah dalam hierarki proses.

4.5.2. Pengecualian Internal

Daripada menghentikan seluruh proses, kita dapat menangani pengecualian dengan menghentikan aktivitas tertentu yang menyebabkan pengecualian tersebut. Selanjutnya, dapat dijalankan prosedur pemulihan untuk mengembalikan proses ke keadaan konsisten dan melanjutkan proses. Jika tidak memungkinkan, baru kemudian proses dihentikan. BPMN menyediakan *error event* untuk menangani skenario ini.



Gambar 4. 14 Menggunakan *terminate events* untuk memberi sinyal penghentian proses yang abnormal



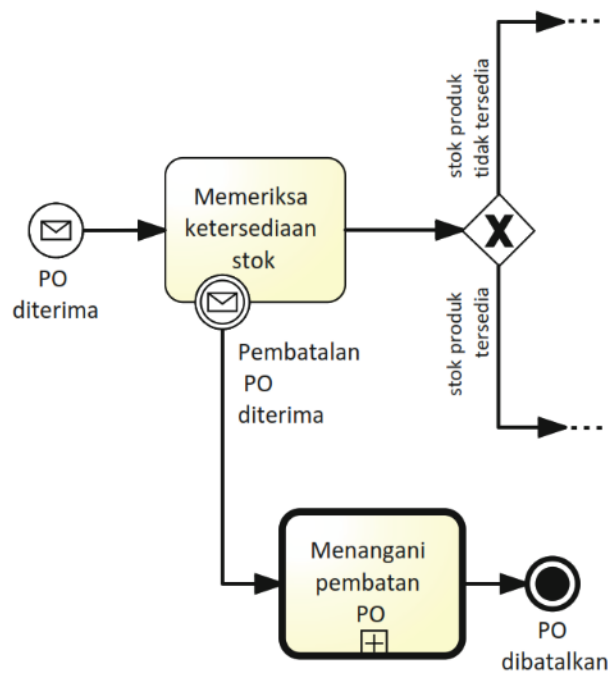
Gambar 4. 15 Model kejadian kesalahan pengecualian internal

End error event digunakan untuk menghentikan sub-proses yang melingkupinya dan melempar pengecualian. Pengecualian ini kemudian ditangkap oleh *intermediate catching error event* yang ditempelkan pada batas sub-proses yang sama. *Boundary event* ini akan memicu prosedur pemulihan melalui alur keluar yang disebut *exception flow*.

Error event digambarkan dengan simbol petir. Sesuai konvensi BPMN, simbol petir kosong digunakan untuk event penangkap (*catching*) dan penuh untuk event pelempar (*throwing*).

Contoh penggunaannya ditunjukkan dalam Gambar 4.15 pada konteks proses *order-to-cash*. Jika terjadi pengecualian karena kehabisan stok, aktivitas pengadaan bahan baku dihentikan dan prosedur pemulihan dimulai, misalnya memberi tahu pelanggan sebelum proses dihentikan.

Secara semantik *token*, saat *end error event* dilempar, semua *token* di sub-proses tersebut dihapus (menghentikannya) dan satu *token* dikirim melalui *exception flow* dari *boundary event*. Tidak ada batasan elemen model yang digunakan dalam *exception flow* untuk memodelkan prosedur pemulihan. Umumnya, *exception flow* diakhiri dengan *end terminate event* atau dikembalikan ke alur normal jika pengecualian berhasil ditangani.



Gambar 4. 16 *Boundary events* menangkap *external events* yang dapat terjadi selama suatu aktivitas

4.5.3. Pengecualian Eksternal

Pengecualian juga dapat disebabkan oleh kejadian eksternal selama aktivitas berlangsung. Misalnya, saat memeriksa ketersediaan stok dalam pesanan pembelian, penjual dapat menerima pembatalan pesanan dari pelanggan. Dalam kasus ini, pemeriksaan stok harus dihentikan dan pembatalan pesanan harus ditangani.

Skenario seperti ini disebut *unsolicited exceptions* karena berasal dari luar proses. Ini dapat ditangkap dengan menempelkan *intermediate catching message event* pada batas aktivitas, seperti pada Gambar 4.16.

Secara semantik, ketika *message event* ini dipicu, token dihapus dari aktivitas yang melingkupinya, menyebabkan aktivitas tersebut dihentikan, dan *token* dialirkan ke *exception flow* untuk menjalankan prosedur pemulihan.

Sebelum menggunakan *boundary event*, perlu diidentifikasi cakupan (*scope*) di mana proses harus responsif terhadap *event* tersebut. Misalnya, dalam proses *order-to-cash*, permintaan pembatalan pesanan hanya dapat ditangani selama pelaksanaan tugas “Memeriksa ketersediaan stok”. Kadang cakupan melibatkan beberapa aktivitas, maka dapat dikemas dalam *sub-proses* dan *event* ditempelkan pada batas sub-proses tersebut.

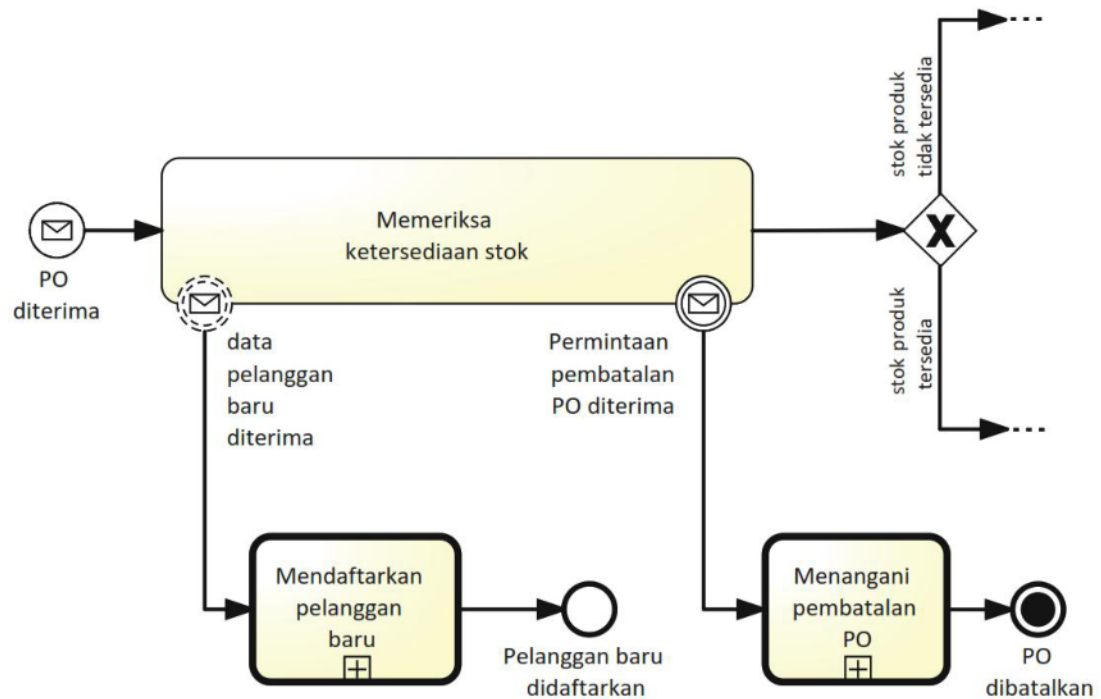
4.5.4. Waktu Habis Aktivitas (Timeout)

Jenis pengecualian lainnya adalah penghentian aktivitas yang memakan waktu terlalu lama. Untuk memodelkan bahwa suatu aktivitas harus selesai dalam jangka waktu tertentu (misalnya, persetujuan dalam 24 jam), kita dapat menempelkan *intermediate timer event* pada batas aktivitas. Timer ini aktif saat aktivitas dimulai. Jika timer menyala sebelum aktivitas selesai, maka aktivitas akan dihentikan. Dengan kata lain, timer pada batas aktivitas berfungsi sebagai *timeout*.

4.5.5. Event Non-Interuptif dan Pengecualian Kompleks

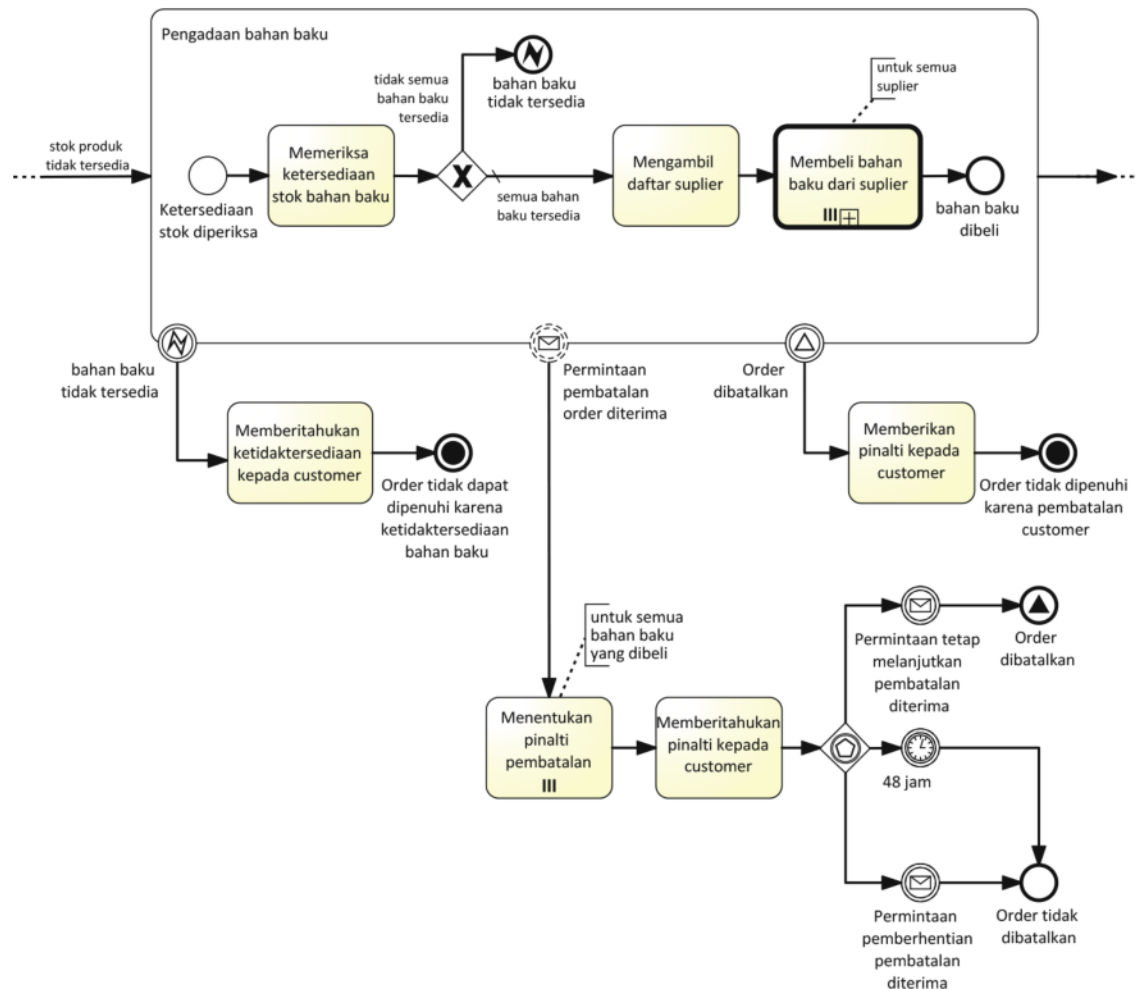
Dalam beberapa kasus, kejadian eksternal selama suatu aktivitas tidak perlu menghentikan aktivitas tersebut, namun tetap harus memicu prosedur. Misalnya, dalam proses *order-to-cash*, pelanggan dapat mengirim permintaan pembaruan data saat pemeriksaan stok. Data pelanggan harus diperbarui tanpa menghentikan pemeriksaan stok.

Untuk menunjukkan bahwa *event* batas tersebut tidak menghentikan aktivitas, digunakan garis tepi putus-putus ganda, seperti pada Gambar 4.17.



Gambar 4. 17 *Non-interrupting boundary events* menangkap *external events* yang terjadi selama suatu aktivitas dan memicu prosedur paralel tanpa mengganggu aktivitas yang melingkupinya

Event non-interrupting dapat digunakan untuk memodelkan penanganan pengecualian yang lebih kompleks. Misalnya, jika pelanggan mengajukan pembatalan saat proses pengadaan bahan baku, kita tangkap permintaan ini dengan *non-interrupting boundary message event*, lalu tentukan penalti berdasarkan bahan yang sudah dipesan. Informasi ini disampaikan kepada pelanggan yang kemudian bisa memutuskan dalam 48 jam apakah akan membatalkan atau tidak (lihat Gambar 4.18).



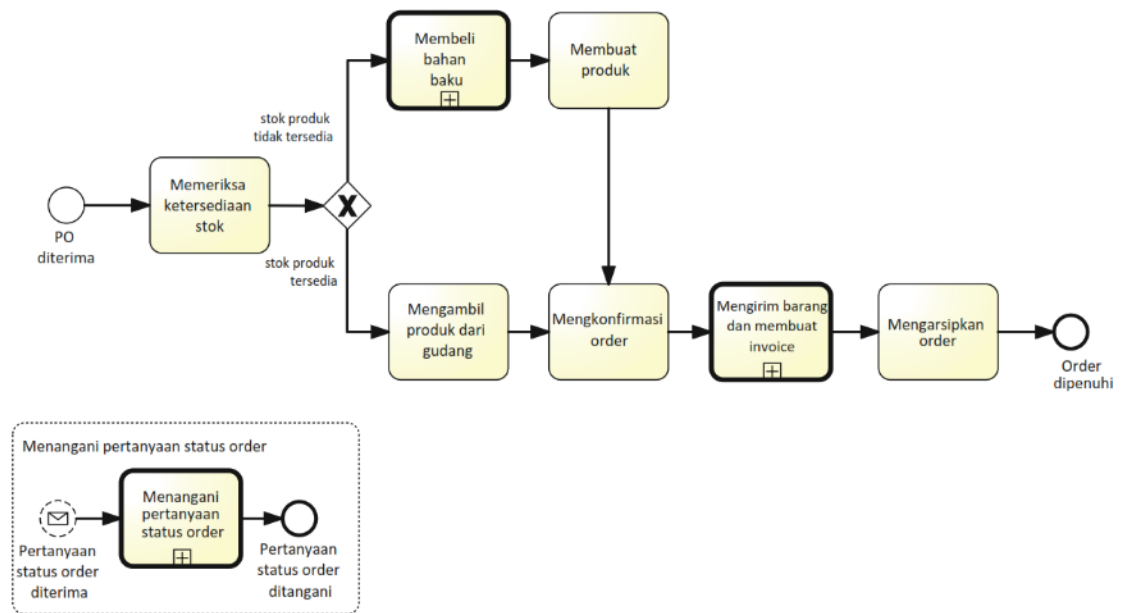
Gambar 4. 18 *Non-interrupting events* dapat digunakan dalam kombinasi dengan *signal events* untuk memodelkan skenario exception handling yang kompleks

Jika pembatalan dilanjutkan, kita lempar *end signal event*. *Signal event* (dengan simbol segitiga) menyiarkan sinyal ke seluruh proses yang memiliki *catching signal event* dengan label yang sama. *Signal* berbeda dari *message* karena tidak memiliki target spesifik, sedangkan *message* memiliki sumber dan target tertentu.

4.5.6. Sub-proses Event (Event Sub-process)

Alternatif dari *boundary event* adalah *event sub-process*. *Event sub-process* dimulai oleh *event* yang seharusnya ditempelkan di batas aktivitas, dan mencakup prosedur yang dipicu oleh *event* tersebut.

Perbedaan penting dengan *boundary event* adalah *event sub-process* tidak harus merujuk ke aktivitas tertentu, tetapi dapat menangani *event* selama eksekusi seluruh proses. Misalnya, kapan saja selama proses *order-to-cash*, pelanggan bisa mengirim pertanyaan tentang status pesanan. Ini bisa ditangani dengan *event sub-process* (lihat Gambar 4.19).



Gambar 4. 19 *Event sub-processes* dapat digunakan sebagai pengganti *boundary events* dan untuk menangkap peristiwa yang dilemparkan dari luar cakupan subproses tertentu.

Event sub-process digambarkan dalam kotak putus-putus dengan sudut membulat. Bisa diletakkan di dalam sub-proses yang diperluas atau proses tingkat atas. Jika *start event* tidak menghentikan proses utama, maka digambarkan dengan garis tepi putus-putus tunggal. Semua aturan sintaksis sub-proses berlaku, kecuali bahwa *boundary event* tidak dapat didefinisikan pada *event sub-process*.

Event sub-process bisa juga digunakan sebagai sub-proses yang disembunyikan (*collapsed*), dan event pemicunya ditampilkan di pojok kiri atas.

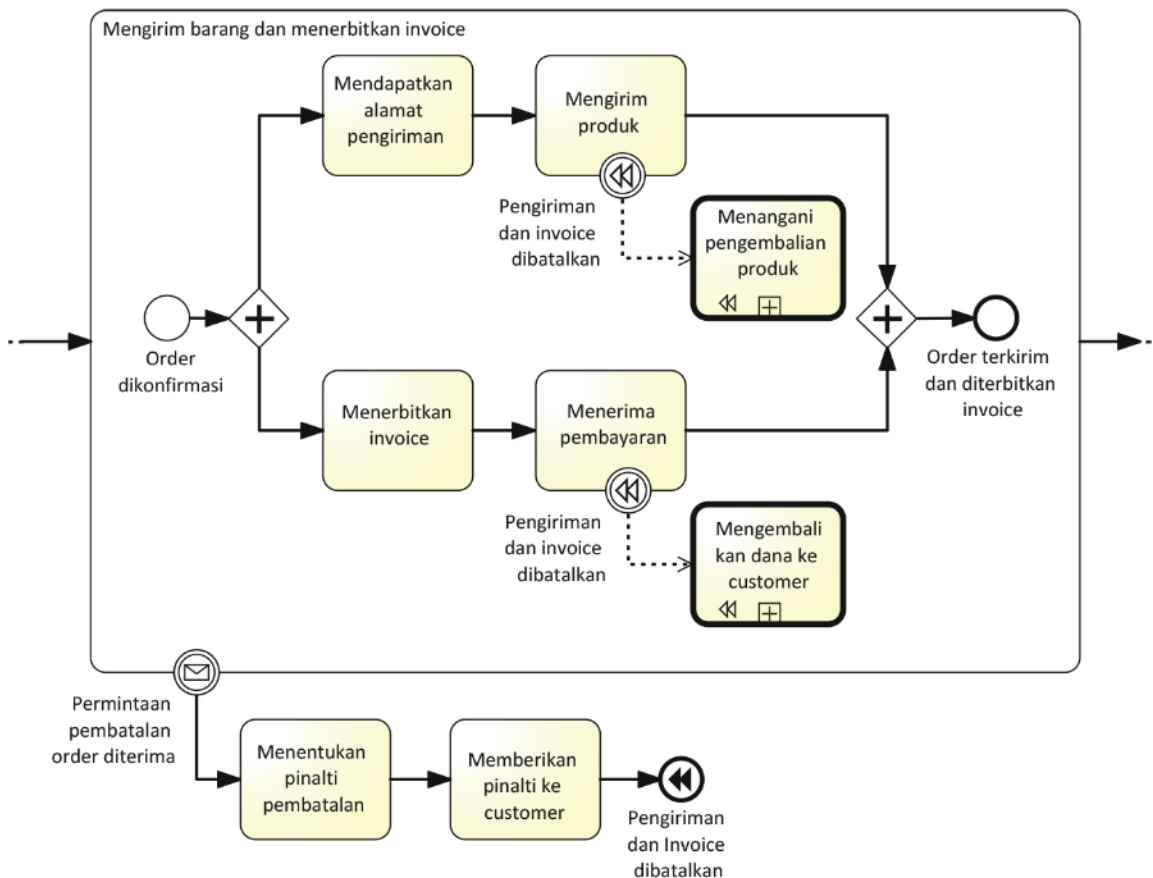
Kapan menggunakan *event sub-process* atau *boundary event*?

Event sub-process bersifat mandiri dan harus diakhiri dengan *end event*, sehingga tidak dapat disambungkan kembali ke alur utama proses. Namun, mereka bisa digunakan ulang sebagai proses global dan dapat didefinisikan di tingkat proses keseluruhan. Gunakan *event sub-process* jika event bisa terjadi kapan saja atau jika prosedurnya akan digunakan ulang. Untuk kasus lain, *boundary event* lebih tepat karena bisa disambungkan kembali ke alur utama.

4.5.7. Kompensasi Aktivitas

Sebagai bagian dari prosedur pemulihan, kadang kita perlu membatalkan satu atau lebih langkah yang telah diselesaikan sebelumnya karena pengecualian yang terjadi. Hasil dari langkah tersebut, dan dampak ikutannya, mungkin tidak diinginkan dan harus

dibalik. Operasi ini disebut kompensasi, yang berusaha mengembalikan proses ke keadaan semula.



Gambar 4. 20 Kompensasi untuk pengiriman dan pembayaran

Misalnya, dalam sub-proses pengiriman dan penagihan pada proses *order-to-cash*, jika terjadi pembatalan pesanan, maka setelah penalti dikomunikasikan ke pelanggan, kita perlu membalik dampak pengiriman dan pembayaran. Jika produk sudah dikirim, maka harus dikembalikan. Jika pembayaran sudah diterima, maka harus dikembalikan kepada pelanggan.

Kompensasi dimodelkan dengan kompensasi *handler*, terdiri dari:

- *Throwing compensate event* (event dengan simbol *rewind*),
- *Catching intermediate compensate event*,
- Aktivitas kompensasi (seperti “Mengembalikan dana ke customer”).

Throwing compensate event digunakan di dalam prosedur pemulihan untuk memulai kompensasi dan dapat berupa *event intermediate* atau *end*. *Catching event* ditempelkan pada aktivitas yang perlu dikompensasi (misalnya “Mengirim produk” dan “Menerima pembayaran”), dan akan memicu aktivitas kompensasi.

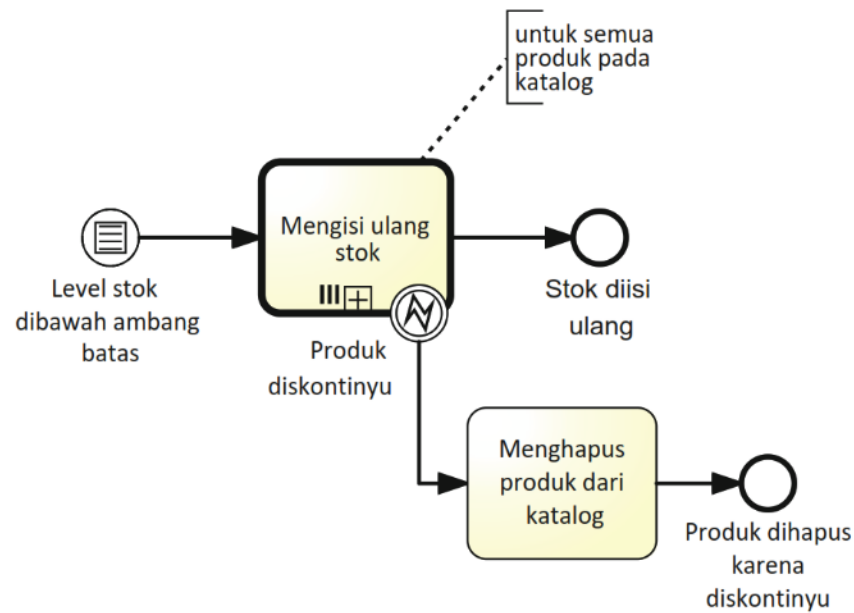
Aktivitas kompensasi tidak boleh memiliki alur keluar, dan dapat berupa sub-proses jika prosedurnya kompleks. Kompensasi hanya efektif jika aktivitas yang dikompensasi sudah selesai. Setelah semua kompensasi dilakukan, proses dilanjutkan dari setelah *throwing event*, kecuali *event* tersebut adalah *end event*.

Jika kompensasi mencakup seluruh proses, kita bisa menggunakan *event sub-process* dengan *start compensate event*.

4.6. Proses dan Aturan Bisnis

Aturan bisnis merupakan implementasi dari kebijakan atau praktik organisasi. Sebagai contoh, dalam sebuah toko daring, pelanggan dengan status platinum memperoleh diskon sebesar 20% untuk setiap pembelian di atas 10 Juta. Aturan bisnis dapat muncul dalam berbagai bentuk di dalam model proses. Kita telah melihat bahwa aturan ini dapat dimodelkan dalam bentuk aktivitas pengambilan keputusan serta dalam kondisi aliran keluar dari sebuah pemisah (X)OR. Pilihan ketiga adalah penggunaan elemen khusus dalam BPMN yang disebut *conditional event* (peristiwa bersyarat). *Conditional event* akan mengaktifkan aliran keluar darinya ketika aturan bisnis yang bersangkutan telah terpenuhi. Peristiwa bersyarat, yang ditandai dengan simbol lembaran berpola garis, dapat digunakan sebagai *start event* maupun *intermediate catching event*, termasuk setelah *event-based gateway*, atau dapat pula ditempelkan pada batas suatu aktivitas.

Perbedaan antara *intermediate conditional event* dan kondisi pada suatu aliran (*flow condition*) adalah bahwa kondisi pada aliran hanya diuji satu kali, dan jika tidak terpenuhi, maka aliran tersebut tidak diambil (aliran lain atau aliran *default* akan diambil sebagai gantinya). Sementara itu, *conditional event* akan terus-menerus diuji hingga aturan yang terkait dipenuhi. Dengan kata lain, *token* akan tetap tertahan sebelum peristiwa tersebut hingga aturan tersebut terpenuhi.



Gambar 4. 21 Permintaan pengisian ulang dipicu setiap kali level stok turun di bawah ambang batas

Pada contoh di Gambar 4.21, dapat diamati penggunaan *error event* pada batas aktivitas *multi-instance*. Peristiwa ini hanya akan menginterupsi *instance* aktivitas yang berkaitan dengan produk tertentu yang diskontinyu, yakni *instance* tempat *error event* tersebut muncul. Semua *interrupting boundary event* lainnya—seperti *message*, *timer*, *signal*, dan *conditional*—akan menginterupsi seluruh instansi dari suatu aktivitas *multi-instance*.

4.7. Studi kasus

1. Silahkan berdiskusi kelompok untuk kasus di bawah ini (semua anggota kelompok harus berkontribusi dan dituliskan pada laporan studi kasus)

Sebuah bank menerima pengajuan kartu kredit secara online. Proses dimulai saat formulir pengajuan diterima dari pelanggan.

Pemeriksaan awal dilakukan untuk memastikan data lengkap dan dokumen pendukung terlampir. Jika tidak lengkap, permintaan pengajuan dikembalikan ke pelanggan untuk diperbaiki. Pengembalian ini dapat terjadi berulang kali hingga data dianggap lengkap (pengulangan).

Setelah data lengkap, sistem secara otomatis melakukan pemeriksaan riwayat kredit dan verifikasi data identitas secara paralel. Jika salah satu dari hasil

pemeriksaan ini menunjukkan kegagalan, pengajuan ditolak (pengulangan paralel + XOR decision).

Jika kedua verifikasi berhasil, proses dilanjutkan ke tahap interview telepon dengan analis kredit. Namun, pelanggan dapat membatalkan pengajuan kapan saja sebelum interview dilakukan (event-based cancellation).

Saat wawancara berlangsung, pelanggan dapat mengirimkan permintaan untuk mengubah data pekerjaan mereka. Permintaan ini tidak menghentikan wawancara, tetapi hanya memicu pembaruan pada data pelanggan (non-interrupting event).

Apabila sistem mendeteksi bahwa proses pengajuan melebihi waktu 5 hari kerja, maka aktivitas evaluasi dipaksa untuk dihentikan dan dilakukan pemberitahuan ke pelanggan (activity timeout).

Jika ditemukan kejanggalan pada saat analisis (misalnya data ganda), proses akan dihentikan secara internal dan dilakukan penyelidikan manual oleh staf fraud detection (internal exception). Jika terbukti tidak ada pelanggaran, proses dilanjutkan.

Jika sistem mendeteksi gangguan server, maka proses akan gagal secara total, semua aktivitas dibatalkan (terminate event).

Setelah seluruh tahapan selesai, kontrak digital dikirimkan ke pelanggan. Proses pengiriman ini dilakukan menggunakan end message event.

Selama proses berjalan, kapan pun pelanggan dapat menanyakan status pengajuan mereka melalui fitur chat. Permintaan ini dapat datang kapan saja dan ditangani oleh sistem tanpa menghentikan proses utama (event sub-process).

Jika pada akhirnya pelanggan memutuskan membatalkan pengajuan setelah kontrak dikirim, maka proses akan menjalankan prosedur kompensasi, yaitu:

Membatalkan kontrak secara digital,

Menghapus data dari sistem,

Memberikan notifikasi pembatalan ke pihak ketiga (jika ada).

Tugas Mahasiswa (Kelompok):

Buatlah model BPMN berdasarkan studi kasus di atas dalam bentuk model fragmen.

Tandai dan beri keterangan bagian model yang menggambarkan:

- Aktivitas loop

- Multi-instance parallel
- Ad-hoc sub-process
- Message/temporal/racing events
- Internal dan external exceptions
- Event sub-process
- Compensation